

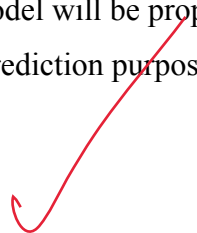
Table of Contents

1.0 Business Understanding	4
2.0 Data Understanding	5
2.1 Overview of the chosen dataset	5
2.2 SNS Pairplot	7
2.3 Boxplot Analysis for features (X Values)	8
2.4 Correlation Between each Variables	9
2.5 Result of Basic Statistic Analysis of each points	9
3.0 Data Preparation	11
3.1 Preparing and preprocessing the dataset (4 Steps)	12
3.2 Computing Percentile Values	15
4.0 Modelling	16
4.1 Logistic Regression	17
4.2 K-Nearest Neighbour (KNN)	19
4.3 Support Vector Machine/ Support Vector Classification (SVM)	24
5.0 Evaluation	29
5.2 Comparing Mean Squared Error of all Models	30
5.3 Comparing Root Mean Squared Error of all Models	31
5.4 Determining the Final Best and Worst Model in overall	32
6.0 Deployment	33
7.0 Conclusion	36
7.1 Advantages and disadvantages of Logistic Regression, KNN and SVM	36
8.0 Reference	37
9.0 Appendix	38
9.1 Dataset	38

1.0 Business Understanding

Hotayi Electronic (M) Sdn. Bhd. is a leading Global Electronics Manufacturing Services company that has been providing a wide range of high value added manufacturing solutions and services since 1992 such as PCB assembly. It is a company that has accumulated technical expertise and experiences in manufacturing core strengths which help the company establish as a leading-edge solution manufacturing provider that is able to meet and exceed customer needs in terms of quality, cost and delivery requirements.

The primary goal and objective of this project is to find the best model by comparing its accuracy, Mean Squared Error and Root Mean Squared Error. To achieve this, 3 different models will be used, which are Logistic Regression, K-Nearest Neighbor (KNN) and Support Vector Machine (SVM). The provided dataset will be used here to train and test the data in order to obtain the best results, which are accuracy, mean squared error and root mean squared error to determine which is the best model. Therefore, the best machine learning model will be proposed and deployed into the Hotayi Electronic (M) Sdn. Bhd for machine failure prediction purposes.



2.0 Data Understanding

use third pov

In this assignment, we have gathered 11538 dataset csv format files from Hotayi Electronic. Each dataset file contained 1 row with 12 columns each. After combining all the dataset files into one new csv file, it now contains about 11538 rows along with 12 columns in the csv file by using the merge feature in Microsoft Excel Spreadsheet. The data type in the new combined csv file contains float64(4), int64(1) and object(7) with memory usage 1.1+ MB. As we analysed manually in the dataset, we found that the most important columns which is known as features are from '(um)Point1' to '(um)Point4' and the target value is the 'Machine result' column where it contains only 2 values which is 'OK' and 'NG'. The other unused column will be continued studied and kept for future usage purposes.

2.1 Overview of the chosen dataset

```
dataset = pd.read_csv('Result_MC2_2021_04.csv')
```

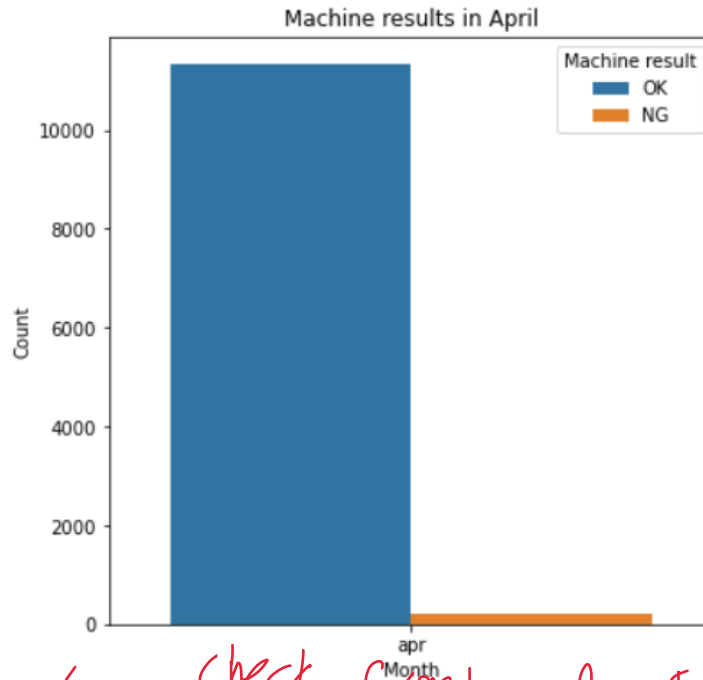
DATA UNDERSTANDING AND PREPROCESSING

```
dataset.head(5)
```

	Part	IC 2D	Lot	Date	Time	Machine	Server result	Machine result	(um)Point1	(um)Point2	(um)Point3	(um)Point4
0	791-00487	22483	G02	04/04/2021	12:19:42 AM	PA05-1901	OK	OK	-1098.2	-1258.6	-1356.9	-1393.9
1	791-00487	22484	G02	04/04/2021	12:19:42 AM	PA05-1901	OK	OK	-1315.4	-1201.0	-1294.3	-1332.3
2	791-00487	22485	G02	04/04/2021	12:19:42 AM	PA05-1901	OK	OK	-1343.0	-1204.5	-1353.7	-1497.5
3	791-00487	22486	G02	04/04/2021	12:19:42 AM	PA05-1901	OK	OK	-1572.2	-1098.6	-1318.6	-1355.8
4	791-00487	22487	G02	04/04/2021	12:19:42 AM	PA05-1901	OK	OK	-1330.9	-1211.6	-1272.7	-1317.1

Figure(1): Overview of the chosen Dataset

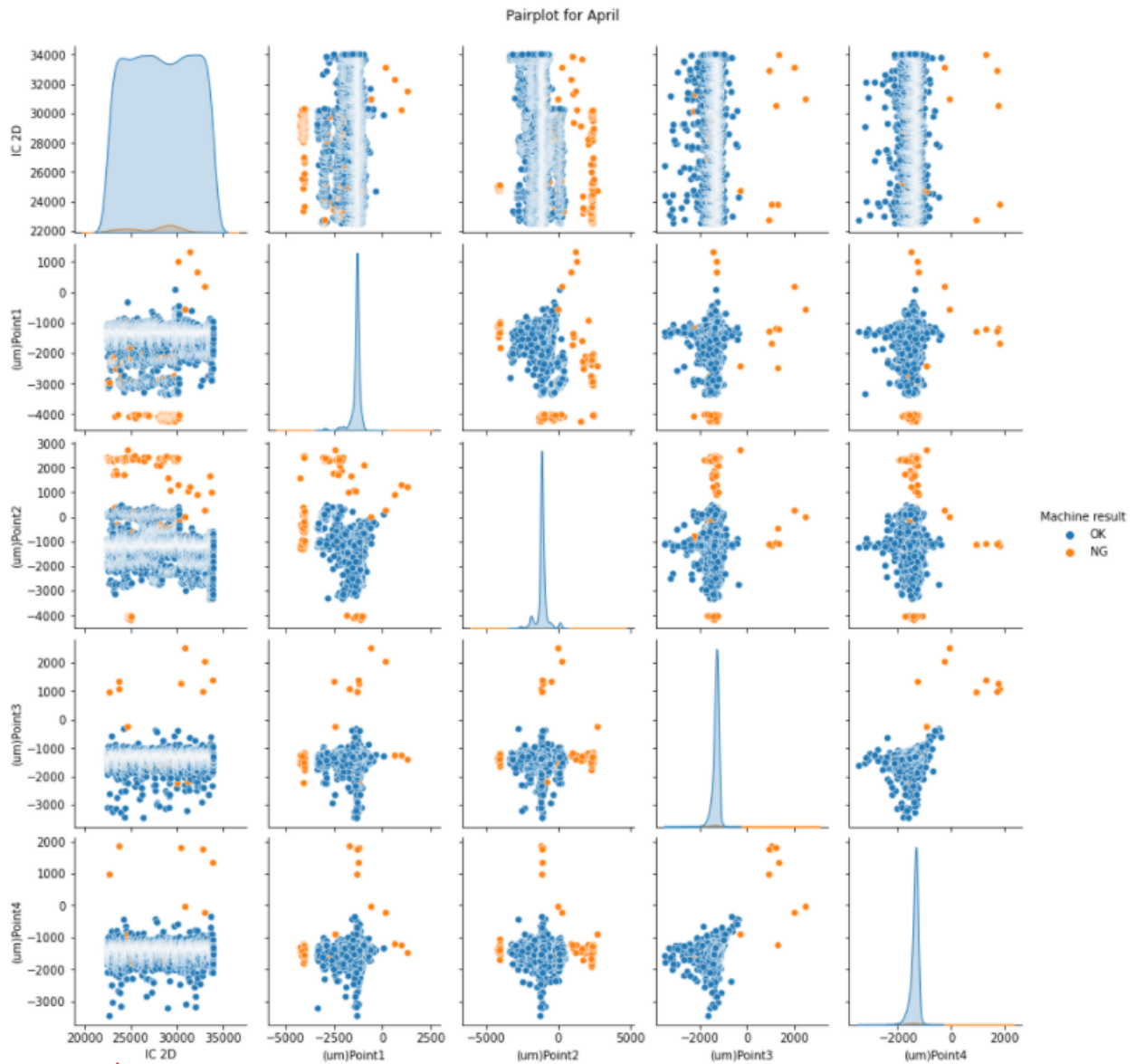
Since our group is limited to 2 members, we choose not to take a very big dataset as this could be a heavy workload to do data preparation and as per our tutor suggestion we proceed to choose one day dataset which is Result_MC2_2021_04/04-04-2021. As mentioned above, that dataset contained a total of 11536 rows of data and fortunately it did not contain any missing values.



Figure(2): Analysis of target Machine result in April Dataset

From **Figure(2)** above, it can be understood that most of the Machine results were 'OK' compared to 'NG'. The ratio of 'OK' machine result to 'NG' machine result is around **1259 : 23** throughout the dataset.

2.2 SNS Pairplot

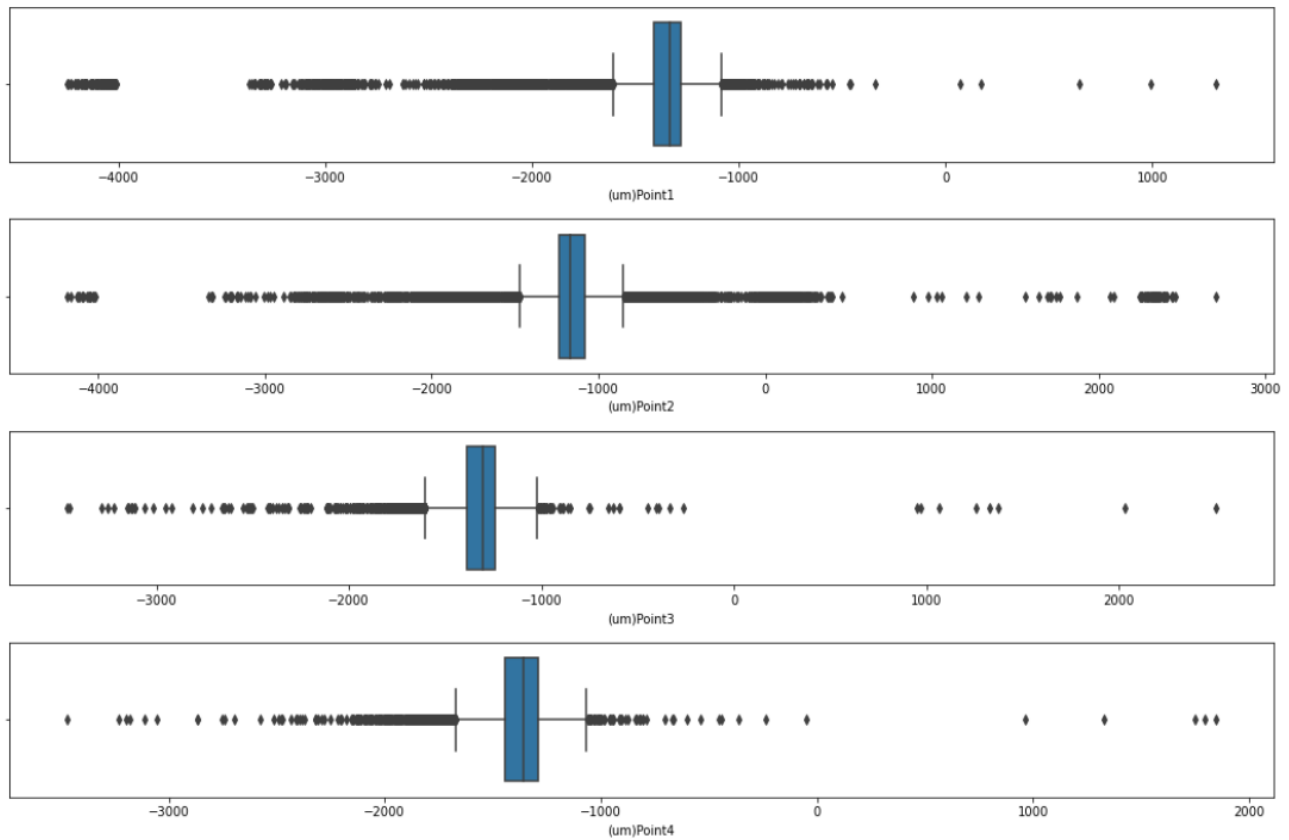


Figure(3): Overall Analysis of target Machine result in April Dataset using SNS Pairplot

From **Figure(3)** above, we can see that most data contained in the Machine result column in the overall April dataset is 'OK' compared to 'NG' using various statistical distributions.

2.3 Boxplot Analysis for features (X Values)

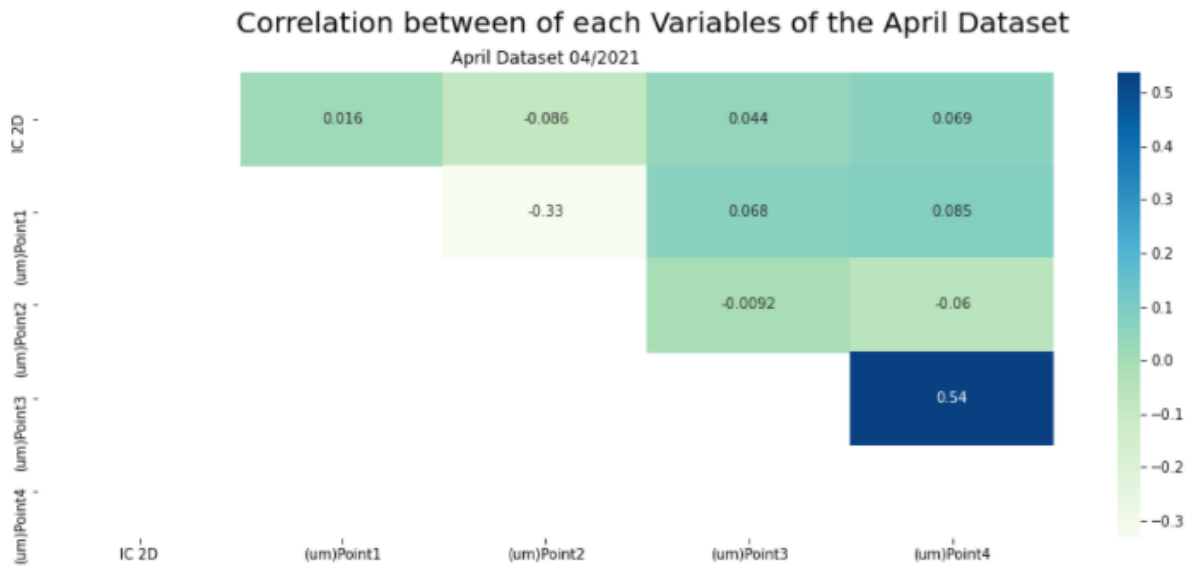
Boxplot for April Features (X Values)



Figure(4): Result of Boxplot for Features in April Dataset

check format, applicable for all figures
From **Figure(4)** above, we can see that (um)Point2 has the most outliers compared to other points in the dataset.

2.4 Correlation Between each Variables



Figure(5): Result of Correlation in April Dataset

From **Figure(5)** above, the (um)Point4 is having a strong positive correlation to (um)Point3 using a lower triangle correlation graph (np.tril).

2.5 Result of Basic Statistic Analysis of each points

```
dataset.describe(exclude=['int64', 'object'])
```

	(um)Point1	(um)Point2	(um)Point3	(um)Point4
count	11538.000000	11538.000000	11538.000000	11538.000000
mean	-1429.090995	-1163.759508	-1329.758572	-1385.816797
std	408.254656	491.326730	181.061365	173.353170
min	-4248.000000	-4188.300000	-3464.100000	-3470.600000
25%	-1411.675000	-1238.100000	-1385.175000	-1444.400000
50%	-1332.600000	-1173.200000	-1304.150000	-1358.350000
75%	-1281.200000	-1082.400000	-1239.625000	-1291.900000
max	1307.400000	2703.300000	2500.800000	1845.000000

Figure(6): Result of Basic Statistic Analysis

Based on **Figure(6)**, there are 2 observations indicating that the columns have extreme outliers. For columns without null/missing values, it can be seen that some columns have median slightly lesser than mean except column (um)Point2 and it may be due to its outliers as mentioned earlier. Secondly, every column without null/missing values has a significant difference between the value of 3rd quartile (75%) and max value.



use 3rd POV

3.0 Data Preparation

As we proceed to data preparation or also known as data preprocessing, we have selected columns from (um)Point1 to (um)Point4 as features to be measured from different sensors of the machine. We have dropped some unwanted columns which is 'Machine', 'Server result', 'Part', 'Lot' and 'IC 2D' because those columns have repetitive data such as 'PA05-1901', 'OK', '791-00487', 'G02' respectively. For the column 'IC 2D' however has been dropped too because it returns no meaningful data which contributes to machine learning. The columns of 'Date' and 'Time' have also dropped since we are not going to reap any useful information in this assignment.

The column 'Machine result' will be converted into value 1 and 0 for 'OK' and 'NG' respectively while 1 means OK and 0 means NG using (.replace(mapping)). One advantage of the chosen dataset is that there are no missing values so replacing missing values is not necessary since it will not affect the accuracy of different machine learning models.

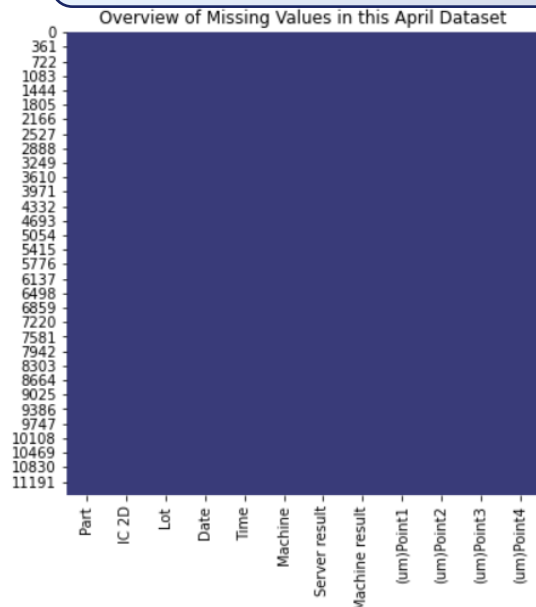


3.1 Preparing and preprocessing the dataset (4 Steps)

Step 1 - Identifying Missing Values

```
#check for missing values
dataset.isnull().sum()
```

```
#heatmap for missing values
plt.figure(figsize=(10,8))
sns.heatmap(dataset.isnull(), cmap="tab20b", cbar=False).set title('Overview of Missing Values in this April Dataset')
plt.show()
```



Figure(7): Checking for Missing Values

Based on the **Figure(7)**, we have found that the chosen dataset has no missing values so replacing missing values is not necessary since it will not affect the accuracy of different machine learning models. We used heatmap to analyse the missing values.

Step 2 - Dropping Unwanted Columns and Splitting the Dataset into train and test set

```
#Dropping Unwanted columns and splitting into train and test data (70:30)
dataset_train, dataset_test = train_test_split(dataset.drop(columns=['Part', 'IC 2D', 'Lot', 'Date', 'Time', 'Machine', 'Server result']), test_size=0.3, random_state=42)
```

Figure(8): Dropping Unwanted Columns and Splitting the Dataset

Based on the **Figure(8)**, we have dropped some unwanted columns which is 'Machine', 'Server result', 'Part', 'Lot' and 'IC 2D' because those columns have repetitive data such as

'PA05-1901', 'OK', '791-00487', 'G02' respectively. For the column 'IC 2D' however has been dropped too because it returns no meaningful data which contributes to machine learning. The columns of 'Date' and 'Time' have also dropped since we are not going to reap any useful information in this assignment.

We split the dataset into a training set and testing set respectively with a ratio of 70:30 and named 'dataset_train' and 'dataset_test' using 'from sklearn.model_selection import train_test_split'.

Step 3 - Separating the train set and test set into X and y respectively

```
def dataset_seperator(dataset):  
    X = dataset.drop("Machine result", axis=1)  
    y = dataset["Machine result"]  
    return X, y  
  
dataset_X_train, dataset_y_train = dataset_seperator(dataset_train)  
dataset_X_test, dataset_y_test = dataset_seperator(dataset_test)  
  
#This function is to show the columns and shape of the both train and test  
#of 'X' dataset (Features) to check whether the unwanted columns have dropped or not  
def print_feature(X_train, X_test):  
    print("X train shape:", X_train.shape, "\X test shape:", X_test.shape)  
    print(X_train.shape, X_test.shape)  
  
print("dataset:")  
print_feature(dataset_X_train, dataset_X_test)
```

```
dataset:  
X train shape: (8076, 4) \X test shape: (3462, 4)  
4 features: ['(um)Point1', '(um)Point2', '(um)Point3', '(um)Point4']
```

Figure(9): Separating the train set and test set into X and y respectively

Based on the **Figure(9)**, we created a function named dataset_seperator where it mainly separates the training set and testing set into X and y respectively. The X contains the features of the dataset which is from (um)Point1 to (um)Point4 where else the y contains the target of the dataset which is 'Machine result'. As we can see, the X train set has 8076 rows with 4 columns and the X test set has 3462 rows with 4 columns.

Step 4 - Converting Column 'Machine Result' into values of 0 and 1

```
#Change machine result from string to binary 0 and 1

#dataset['Machine result'] = dataset['Machine result'].astype(object)

dataset_y_train = pd.DataFrame(dataset_y_train, columns=['Machine result'])
dataset_y_test = pd.DataFrame(dataset_y_test, columns=['Machine result'])

mapping = {"OK":1, "NG":0}
dataset_y_train["Machine result"] = dataset_y_train["Machine result"].replace(mapping)
dataset_y_test["Machine result"] = dataset_y_test["Machine result"].replace(mapping)

dataset_y_train.head(5)

#dataset_y_test(5)
```

REVIEWED

By TARUC at 12:40 pm, Aug 08, 2023

Machine result	
9312	1
3319	1
9527	1
1739	1
230	1

Figure(10): Converting Column 'Machine Result' into values of 0 and 1

Based on the **Figure(10)**, we know that the Column 'Machine Result' has string values such as 'OK' and 'NG' in the dataset. Therefore, we converted the values to 0 and 1, where 1 means OK and 0 means NG using (.replace(mapping)) for both the train and test set of y.



3.2 Computing Percentile Values

PERCENTILE

```
#computing percentile values
def percentile_displayer(datasets):
    percentile = np.zeros(shape=(0,2))
    for feature in datasets.columns:
        ten = round(datasets[feature].quantile(0.10), 2)
        nine = round(datasets[feature].quantile(0.90), 2)

        print(feature, '- Percentile of 10th: ', "%8.2f" % (ten))
        percentile = np.append(percentile, np.array([feature, ten]), axis=1)

    print()
    return percentile

#Print percentile values for both training and testing dataset
print('Percentile in April training set: ')
dataset_percentile_train = percentile_displayer(dataset_X_train)
print("\n")
print('Percentile in April testing set: ')
dataset_percentile_test = percentile_displayer(dataset_X_test)
```

Percentile in April training set:

(um)Point1 - Percentile of 10th:	-1676.70		- Percentile of 90th:	-1207.15
(um)Point2 - Percentile of 10th:	-1611.50		- Percentile of 90th:	-889.80
(um)Point3 - Percentile of 10th:	-1500.10		- Percentile of 90th:	-1189.20
(um)Point4 - Percentile of 10th:	-1571.70		- Percentile of 90th:	-1239.20

Percentile in April testing set:

(um)Point1 - Percentile of 10th:	-1643.33		- Percentile of 90th:	-1197.70
(um)Point2 - Percentile of 10th:	-1481.60		- Percentile of 90th:	-894.39
(um)Point3 - Percentile of 10th:	-1490.13		- Percentile of 90th:	-1188.00
(um)Point4 - Percentile of 10th:	-1578.23		- Percentile of 90th:	-1240.43

Figure(11): Finding the percentile values

Based on the **Figure(11)**, we made a simple data analysis of percentile computation for uses like to give a number that describes the value that a given percent of the values are lower than. Meaning that each training set and testing set of April X feature values, we computed to find each 10th and 90th percentile value respectively. The purpose for this percentile computation is for a set of data that needs to be broken into digestible chunks and used to interpret test scores of the dataset which makes it more convenient when fitting those dataset into a model.

4.0 Modelling

why? reference?

As we proceed to the modelling phase, we have chosen and listed a class of models by using the training and testing set of 70:30 splitting ratio by using 'from sklearn.model_selection import train_test_split'. These are the chosen models to be tested:

1. Logistic Regression

2. K-Nearest Neighbour Classifier (KNN)

3. Support Vector Machine (SVM)

We fitted the models into the following datasets below to predict the machine results, by using the machine result which converted the values to 0 and 1, where 1 means OK and 0 means NG using (.replace(mapping)) for both the train and test set of y. Some output scores of the models will be shown below. The respective classification report can be found below too and in our ipynb file.



4.1 Logistic Regression

In this part of modelling, we are using one of three selected models, Logistic Regression to get the result. First, we fit the both dataset which are dataset_X_train and dataset_y_train into the Logistic Regression model by using `sklearn.linear_model.logisticregression` imported from `sklearn` library. Then, we use `predict` method on the train and test data to obtain the score. The score of test data is 0.9905 meanwhile the training set's score is 0.9920.

```
#Fit the model
logreg = LogisticRegression()
logreg.fit(dataset_X_train, dataset_y_train)

#predict the test and train result
y_pred = logreg.predict(dataset_X_test)
y_pred_train = logreg.predict(dataset_X_train)

#Check testing accuracy
print("Testing set score Logistic Regression (April) :{:0.4f}".format(accuracy_score(y_pred,dataset_y_test)))
acclrApr = accuracy_score(y_pred,dataset_y_test)

#Check training accuracy
print("\n")
print("Training set score Logistic Regression (April) :{:0.4f}".format(accuracy_score(y_pred_train,dataset_y_train)))
```

Testing set score Logistic Regression (April) :0.9905

Training set score Logistic Regression (April) :0.9920

Figure (12): Steps of fitting the model and scores of test data and train data

After that, we calculate the Mean Squared Error (MSE) and Root Mean Squared Error (RMSE) for this model. The calculated value of MSE is around 0.0095 and RMSE is around 0.0976. Later on, we could use these results to compare in the evaluation part.

```
#Calculate Mean Squared Error and Root Mean Squared Error
print("April:")
print("Mean Squared Error for Logistic Regression April: {}".format(mean_squared_error(dataset_y_test, y_pred)))
print("\n")
print("Root Mean Squared Error for Logistic Regression April: {}".format(mean_squared_error(dataset_y_test, y_pred, squared = False)))
mseLrApr = mean_squared_error(dataset_y_test, y_pred)
rmseLrApr = mean_squared_error(dataset_y_test, y_pred, squared = False)

#push both mse and rmse into respective array
mse_list.append(mseLrApr)
rmse_list.append(rmseLrApr)
```

April:
Mean Squared Error for Logistic Regression April: 0.009532062391681109

Root Mean Squared Error for Logistic Regression April: 0.09763228150402463

Figure (13): Calculating the MSE and RMSE

```

#Classification Report (Making the confusion matrix and calculating accuracy score)
accLrApr = accuracy_score(y_pred,dataset_y_test)

#push the accuracy score of Logistic Regression into accuracy list array
accuracy_list.append(accLrApr)

#print the Accuracy of Logistic Regression
print('Accuracy of Logistic Regression is : ', '{:.2f}%'.format(100 * accLrApr))
print("\n")

#Classification Report
print("Classification Report for Logistic Regression:")
logreg_classification_report = classification_report(dataset_y_test, y_pred)
print(logreg_classification_report)

#Display Graph or Visualize the model
#Confusion matrix
cm7 = confusion_matrix(dataset_y_test,y_pred)
plt.figure()
plot_confusion_matrix(cm7,hide_ticks=True, cmap=plt.cm.PuRd)
plt.title('Logistic Regression Model - Confusion Matrix')
plt.xticks(range(2), ['NG - 0', 'OK - 1'], fontsize=16)
plt.yticks(range(2), ['NG - 0', 'OK - 1'], fontsize=16)
plt.show()

```

REVIEWED
By TARUC at 12:41 pm, Aug 08, 2023

Figure (14): Code of Classification Report and Confusion Matrix

Overall on the Logistic Regression, we obtain the accuracy of 99.05% in this model with precision of 0.88 on '0' and 0.99 on '1'. We also show that the number of support for the machine result on '1' or 'OK' is 3399 and '0' is 63 in the figure (). To be precise, 3394 supports on TP and only 5 supports on FP while FN is 28 and FP is 35.

Accuracy of Logistic Regression is : 99.05%

Classification Report for Logistic Regression:

	precision	recall	f1-score	support
0	0.88	0.56	0.68	63
1	0.99	1.00	1.00	3399
accuracy			0.99	3462
macro avg	0.93	0.78	0.84	3462
weighted avg	0.99	0.99	0.99	3462

Figure (15): Summary of the Classification Report for Logistic Regression

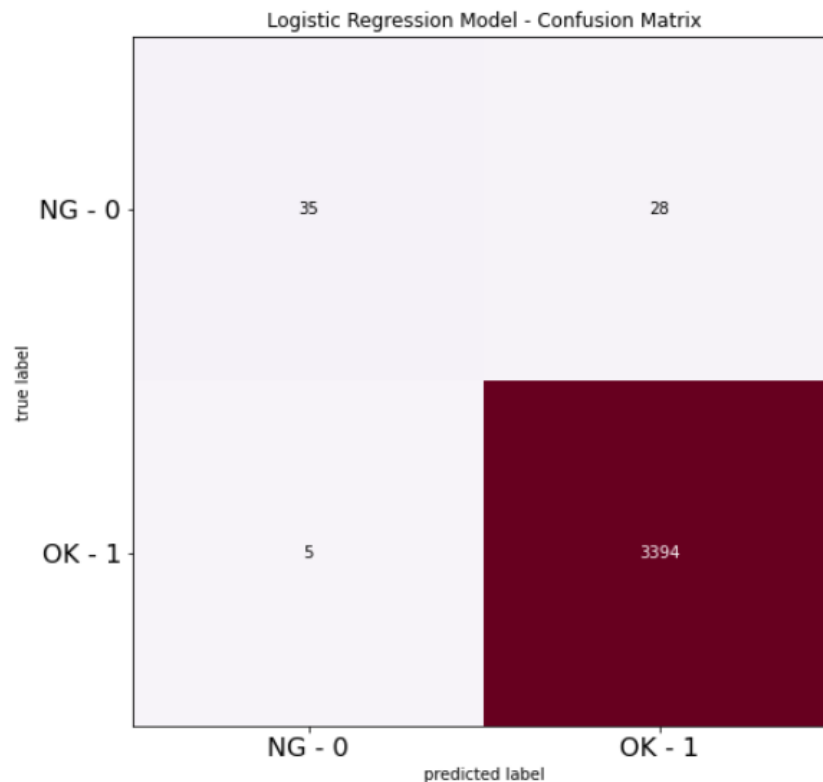


Figure (16): Summary of the Confusion Matrix for Logistic Regression

4.2 K-Nearest Neighbour (KNN)

KNN

```
#Setup arrays to store train and test accuracy for determining the BEST K value
neighbor = np.arange(1,15)
train_dataset = np.empty(len(neighbor))
test_dataset = np.empty(len(neighbor))

for i,k in enumerate(neighbor):
    #setup knn classifier with k neighbours
    knn = KNeighborsClassifier(n_neighbors=k)

    #fit the model
    knn.fit(dataset_X_train,dataset_y_train)

    #compute accuracy on train data
    train_dataset_Accuracy[i] = knn.score(dataset_X_train,dataset_y_train)

    #compute accuracy on test data
    test_dataset_Accuracy[i] = knn.score(dataset_X_test,dataset_y_test)
```

REVIEWED

By TARUC at 12:42 pm, Aug 08, 2023

```
#generate plot
plt.title('KNN Varying number of K neighbors for April')
plt.plot(neighbors, test_accuracy, label='Testing Accuracy')
plt.plot(neighbors, train_accuracy, label='Training Accuracy')
plt.legend()
plt.xlabel('Number of neighbors')
plt.ylabel('Accuracy')
plt.show()
```

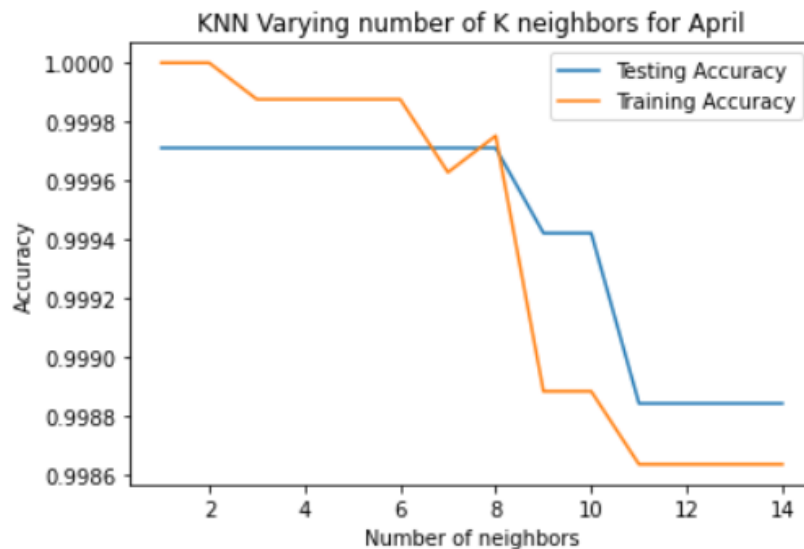


Figure (17): Setup to find the best K value score

From **Figure (17)** above, we implement a method to find the best K value before initializing and fitting into the KNN model. This can ensure that the model can predict and provide higher accuracy easily when testing sets have been used. The graph above shows the value K from 0 to 8 has a better accuracy for the testing set, therefore we choose k = 8 when fitting into the model.

```

#Fit the model k = 8

#Initiate model
knnApr = KNeighborsClassifier(n_neighbors=8)

#Fitting the model using train set
print(knnApr.fit(dataset_X_train,dataset_y_train))

#predict the test and train result
pred_test_y = knnApr.predict(dataset_X_test)
pred_train_y = knnApr.predict(dataset_X_train)

#Check testing accuracy
print("\n")
print("Testing set score KNN (April) with k=8:{0:.4f}" .format(accuracy_score(pred_test_y,dataset_y_test)))
accknnApr = accuracy_score(pred_test_y,dataset_y_test)

#Check training accuracy
print("\n")
print("Training set score KNN (April) with k=8:{0:.4f}" .format(accuracy_score(pred_train_y,dataset_y_train)))

```

REVIEWED

By TARUC at 12:42 pm, Aug 08, 2023

KNeighborsClassifier(n_neighbors=8)

Testing set score KNN (April) with k=8:0.9997

Training set score KNN (April) with k=8:0.9998

Figure (18): Initialise and Fitting into KNN Model

Based on the **Figure (18)** above, we initialise the KNN model using ‘from sklearn.neighbors import KNeighborsClassifier’ and setting k value to 8 which was acquired from the **Figure (17)** above. We fitted the model with dataset_X_train and dataset_y_train and predicted the results using dataset_X_test for testing set and dataset_X_train for training set respectively. We managed to get the final accuracy output for both testing and training set for KNN which is 0.9997 and 0.9998 respectively.



```

#Calculate Mean Squared Error and Root Mean Squared Error
print("April:")
print("Mean Squared Error for KNN April:", mean_squared_error(dataset_y_test, pred_test_y))
print("\n")
print("Root Mean Squared Error for KNN April:", sqrt(mean_squared_error(dataset_y_test, pred_test_y, squared = False)))

mse_knnApr = mean_squared_error(dataset_y_test, pred_test_y)
rmse_knnApr = sqrt(mean_squared_error(dataset_y_test, pred_test_y, squared = False))

#push both mse and rmse into respective array
mse_list.append(mse_knnApr)
rmse_list.append(rmse_knnApr)

```

April:
Mean Squared Error for KNN April: 0.00028885037550548814

Root Mean Squared Error for KNN April: 0.01699559870982744

Figure (19): Computing the Mean Squared Error and Root Mean Squared Error for KNN

Based on the **Figure (19)** above, we found that the Mean Squared Error for KNN is 0.000288 and for the Root Mean Squared Error for KNN is 0.016995 using the testing set.

```

#Classification Report (Making the confusion matrix and calculating accuracy score)
accknnApr = accuracy_score(pred_test_y, dataset_y_test)

#push the accuracy score of KNN into accuracy list array
accuracy_list.append(accknnApr)

#print the Accuracy of KNN
print('Accuracy of KNN is : ', '{:.2f}%'.format(100 * accknnApr))

#Print Classification Report
print("\n")
print("Classification Report for KNN:")
print(classification_report(pred_test_y, dataset_y_test))

#Confusion Matrix
cm = confusion_matrix(pred_test_y, dataset_y_test)
plt.figure()
plot_confusion_matrix(cm, figsize=(12,8), hide_ticks=True, cmap=plt.cm.Blues)
plt.title('KNN Model - Confusion Matrix')
plt.xticks(range(2), ['NG - 0', 'OK - 1'], fontsize=16)
plt.yticks(range(2), ['NG - 0', 'OK - 1'], fontsize=16)
plt.show()

```

Figure (20): Implementation of Accuracy, Classification Report and Confusion Matrix for KNN

Accuracy of KNN is : 99.97%

Classification Report for KNN:

	precision	recall	f1-score	support
0	0.98	1.00	0.99	62
1	1.00	1.00	1.00	3400
accuracy			1.00	3462
macro avg	0.99	1.00	1.00	3462
weighted avg	1.00	1.00	1.00	3462

Figure (21): Accuracy and Classification Report for KNN Model

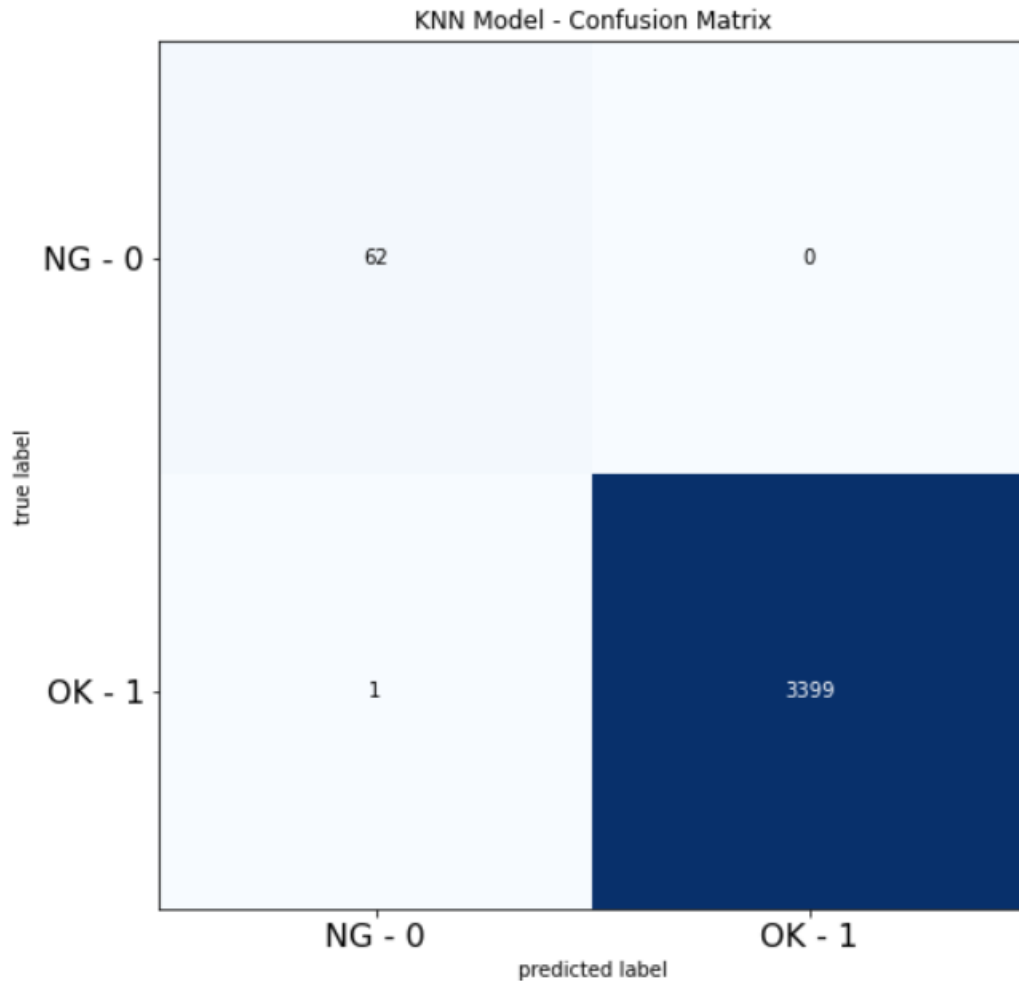


Figure (22): Confusion Matrix for KNN Model

Based on **Figure(21)** above, the accuracy we obtained for KNN is 99.97% with the precision of 0.98 and 1.00 for '0' and '1' respectively in **Figure (21)**. It appears that precision for '1' is overtrained and this might be due to the quantity of the dataset chosen. Moreover, based on **Figure (22)**, the confusion matrix for 'NG' has 62 and 'OK' has 3399 based on the true label and predicted label respectively.

4.3 Support Vector Machine/ Support Vector Classification (SVM)

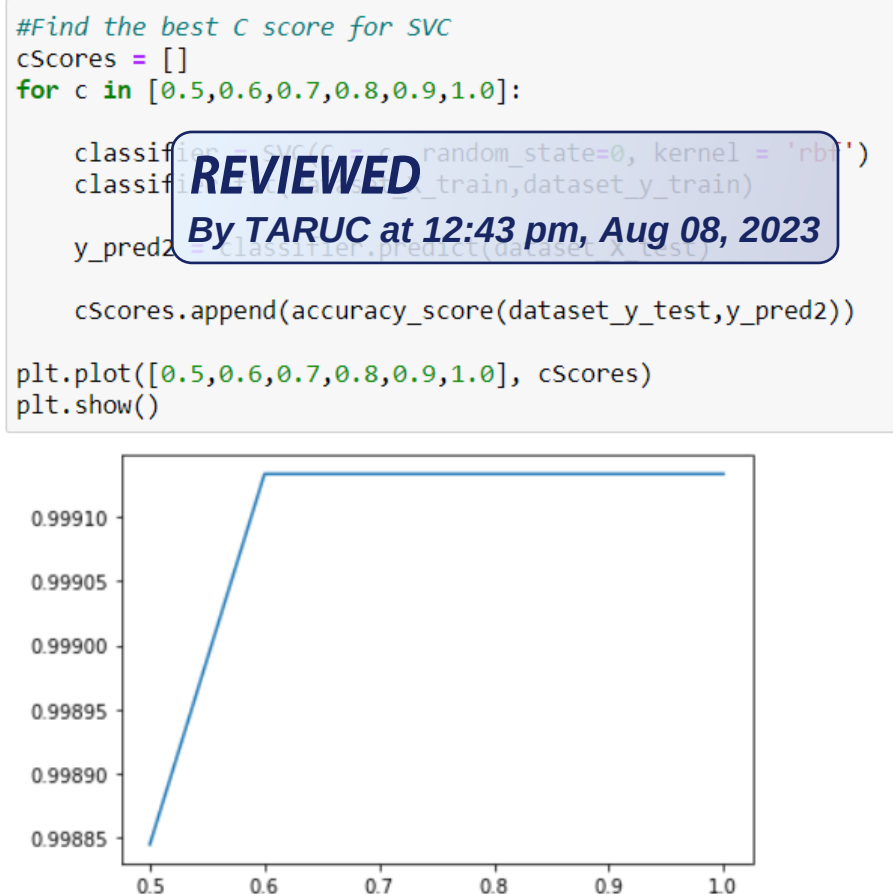


Figure (23): Setup to find the best C score

From **Figure (23)** above, we implement a method to find the best C score before initializing and fitting into the SVM model. This can ensure that the model can predict and provide higher accuracy easily when testing sets have been used. The graph above shows the value C from 0.6

to 1.0 has a better accuracy for the testing set, therefore we choose k = 0.6 when fitting into the SVM model.

```
#Fit the model using C = 0.6
svmClassifier = SVC(C=0.6, gamma=0.01, kernel='rbf')
svmClassifier.fit(dataset_X_train, dataset_y_train)
SVC(C=0.6, gamma=0.01, kernel='rbf')

#predict the test and train result
pred_test_y_svm = svmClassifier.predict(dataset_X_test)
pred_train_y_svm = svmClassifier.predict(dataset_X_train)

#Check testing accuracy
print("Testing set score SVM (April) with k=6:{0:0.4f}".format(accuracy_score(pred_test_y_svm, dataset_y_test)))
accsvmApr = accuracy_score(pred_test_y_svm, dataset_y_test)

#Check training accuracy
print("\n")
print("Training set score SVM (April) with k=6:{0:0.4f}".format(accuracy_score(pred_train_y_svm, dataset_y_train)))

Testing set score SVM (April) with k=6:0.9991

Training set score SVM (April) with k=6:1.0000
```

REVIEWED
By TARUC at 12:43 pm, Aug 08, 2023

REVIEWED
By TARUC at 12:43 pm, Aug 08, 2023

Figure (24): Initialise and Fitting into SVM Model

Based on the **Figure (24)** above, we initialise the SVM model using ‘from sklearn.svm import SVC’ and setting C score to 0.6 which was acquired from the **Figure (23)** above. We fitted the model with dataset_X_train and dataset_y_train and predicted the results using dataset_X_test for testing set and dataset_X_train for training set respectively. We managed to get the final accuracy output for both testing and training set for KNN which is 0.9991 and 1.0000 respectively.



```
#Calculate Mean Squared Error and Root Mean Squared Error SVM
print("April:")
print("Mean Squared Error for SVM April:" , mean_squared_error(dataset_y_test, pred_test_y_svm))
print("\n")
print("Root Mean Squared Error for SVM April:" , mean_squared_error(dataset_y_test, pred_test_y_svm, squared = False))
msesvmApr = mean_squared_error(dataset_y_test, pred_test_y_svm)
rmsesvmApr = mean_squared_error(dataset_y_test, pred_test_y_svm, squared = False)
```

REVIEWED

By TARUC at 12:43 pm, Aug 08, 2023

```
Mean Squared Error for SVM April: 0.0008665511265164644
```

```
Root Mean Squared Error for SVM April: 0.029437240470473188
```

Figure (25): Computing the Mean Squared Error and Root Mean Squared Error for SVM

Based on the **Figure (25)** above, we found that the Mean Squared Error for SVM is 0.000866 and for the Root Mean Squared Error for SVM is 0.02943 using the testing set.

```
#Classification Report (Making the confusion matrix and calculating accuracy score)
accsvmApr = accuracy_score(pred_test_y_svm,dataset_y_test)

#push the accuracy score of SVM into accuracy list array
accuracy_list.append(accsvmApr)

#print the Accuracy of SVM
print('Accuracy of SVM is : ', '{:.2f}%'.format(100 * accsvmApr))

#Print Classification Report
print("\n")
print("Classification Report for SVM:")
print(classification_report(pred_test_y_svm,dataset_y_test))

#Confusion Matrix
cm = confusion_matrix(pred_test_y_svm,dataset_y_test)
plt.figure()
plot_confusion_matrix(cm, figsize=(12,8), hide_ticks=True, cmap=plt.cm.Oranges)
plt.title('SVM Model - Confusion Matrix')
plt.xticks(range(2), ['NG - 0', 'OK - 1'], fontsize=16)
plt.yticks(range(2), ['NG - 0', 'OK - 1'], fontsize=16)
plt.show()
```

Figure (26): Implementation of Accuracy, Classification Report and Confusion Matrix for SVM

Accuracy of SVM is : 99.91%

Classification Report for SVM:

	precision	recall	f1-score	support
0	0.95	1.00	0.98	60
1	1.00	1.00	1.00	3402
accuracy			1.00	3462
macro avg	0.98	1.00	0.99	3462
weighted avg	1.00	1.00	1.00	3462

Figure (27): Accuracy and Classification Report for SVM Model

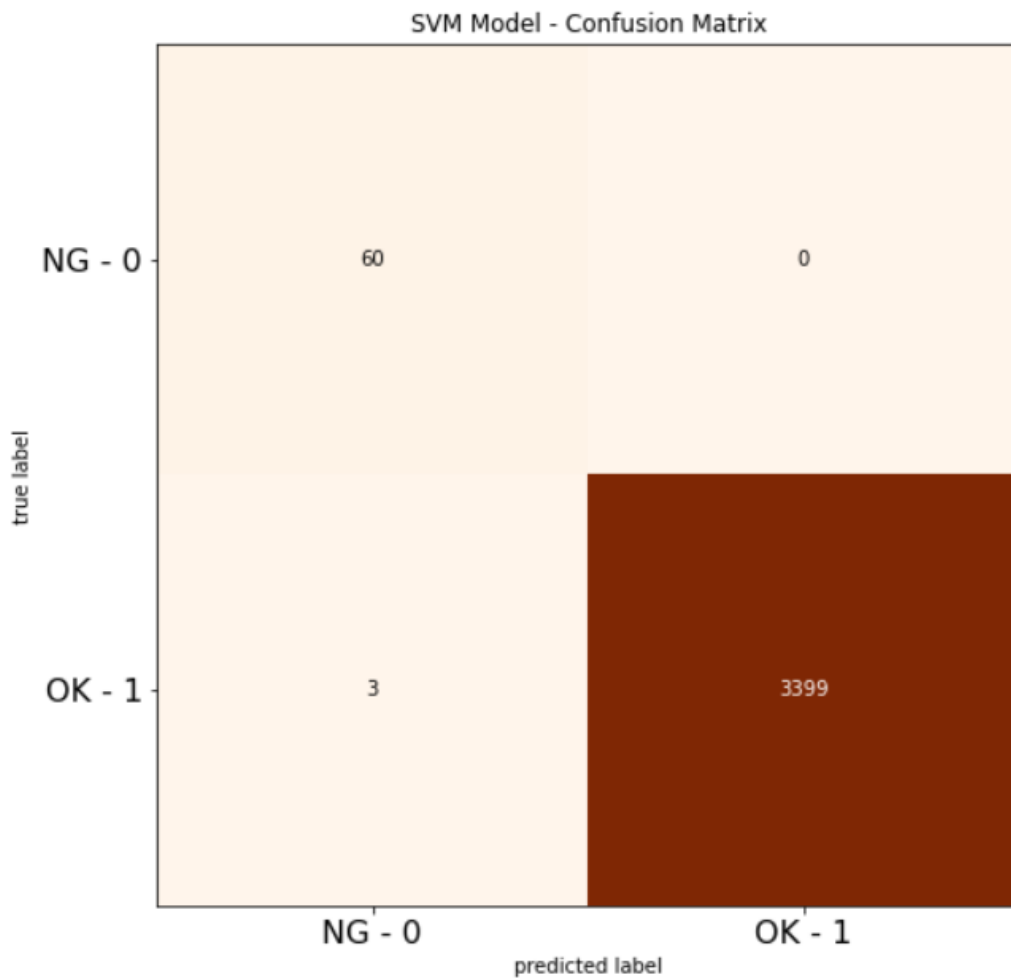


Figure (28): Confusion Matrix for SVM Model

Based on **Figure (27)** above, the accuracy we obtained for SVM is 99.91% with the precision of 0.95 and 1.00 for '0' and '1' respectively in **Figure (27)**. It appears that precision for '1' is overtrained as the KNN model and this might be due to the quantity of the dataset chosen. Moreover, based on **Figure (28)**, the confusion matrix for 'NG' has 60 and 'OK' has 3399 based on the true label and predicted label respectively.



5.0 Evaluation

As we proceed to the evaluation phase, we can now determine whether our primary goal and objective of this project has been successfully achieved, which is to find the best machine learning model by comparing its accuracy, Mean Squared Error and Root Mean Squared Error. As a summary, we confirm that the K-Nearest Neighbour is the best model by its accuracy, Mean Squared Error and Root Mean Squared Error. This KNN model can be proposed and deployed to the Hotayi Electronic (M) Sdn. Bhd for their machine failure prediction purposes.

The prove of evaluation for KNN model will be explained as follows:

5.1 Comparing Accuracies of all Models

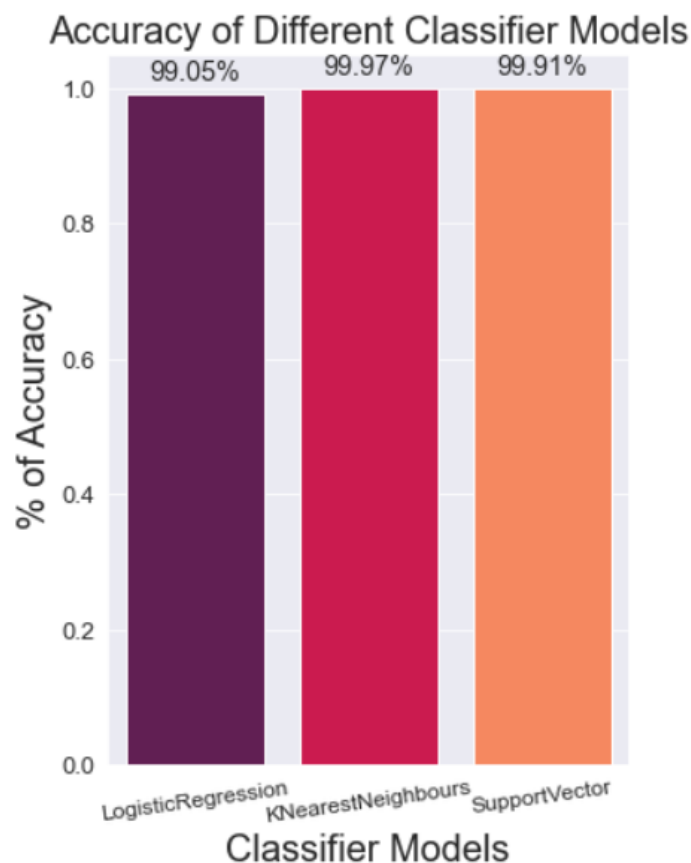


Figure (29): Barplot for comparing Accuracies of all Models

From **Figure (29)** above, the higher the accuracy the better the model will be in predicting the results. Therefore, the KNN model has the highest accuracy which is 99.97% compared to

Logistic Regression and Support Vector Machine. On the other hand the lowest accuracy score is considered as the worst model in predicting the results, therefore the Logistic Regression will be the worst model in this assignment which is 99.05%.

5.2 Comparing Mean Squared Error of all Models

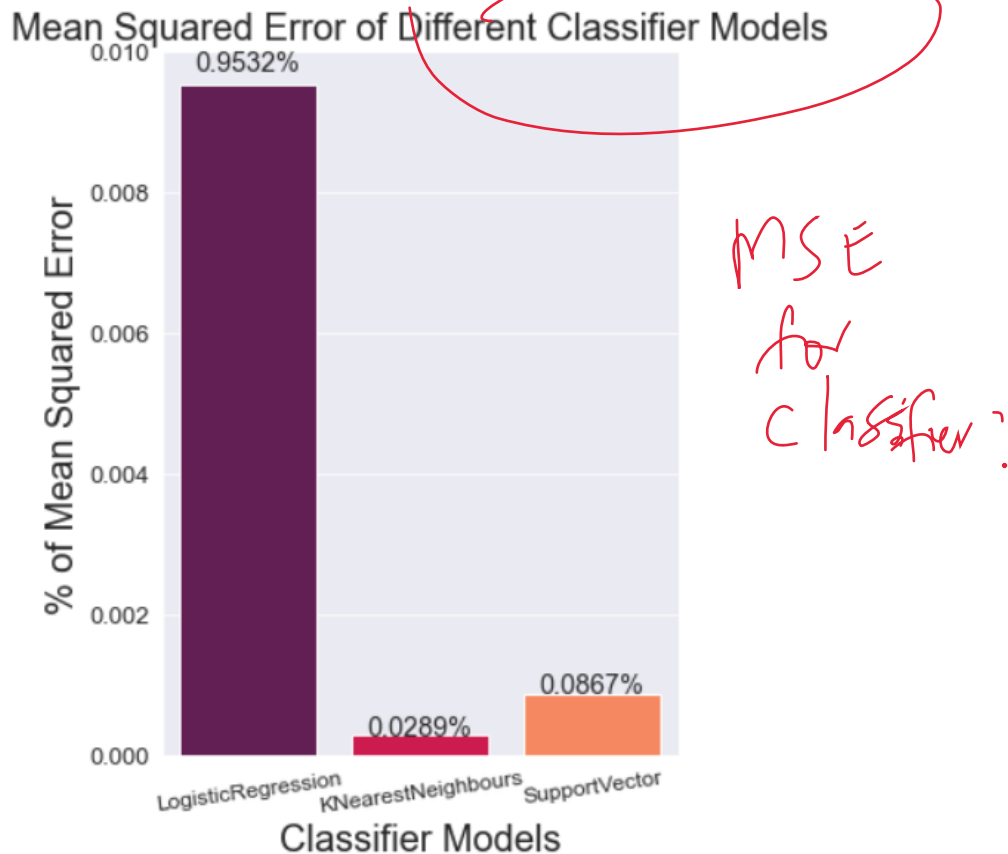


Figure (30): Barplot for comparing Mean Squared Error of all Models

From **Figure (30)** above, the lesser the Mean Squared Error the better the model will be in measuring how close a fitted line is to data points. Therefore, the KNN model has the lowest Mean Squared Error which is 0.0289% compared to Logistic Regression and Support Vector Machine. On the other hand the higher the Mean Squared Error is considered as the worst model in measuring how close a fitted line is to data points, therefore the Logistic Regression will be the worst model in this assignment which is 0.9532%.

5.3 Comparing Root Mean Squared Error of all Models



Figure (31): Barplot for comparing Root Mean Squared Error of all Models

From **Figure (31)** above, the lesser the Root Mean Squared Error the better the model will be in measuring the quality of predictions. Therefore, the KNN model has the lowest Root Mean Squared Error which is 1.6996% compared to Logistic Regression and Support Vector Machine. On the other hand the higher the Root Mean Squared Error is considered as the worst model in measuring the quality of predictions, therefore the Logistic Regression will be the worst model in this assignment which is 9.7632%.



5.4 Determining the Final Best and Worst Model in overall

Determining the best model in terms of accuracy, Mean Squared Error and Root Mean Squared Error

```
#Push all accuracy, Mean Squared Error and Root Mean Squared Error of each model into respective set
accuracies = {'Logistic Regression':acclogApr, 'SVM':accsvmApr}
meanSE = {'Logistic Regression':mseLogApr, 'SVM':msesvmApr}
rmseSE = {'Logistic Regression':rmseLogApr, 'SVM':rmsesvmApr}

#Setting for determining the best model based on accuracy, Mean Squared Error and Root Mean Squared Error
great_acc = max(accuracies, key=accuracies.get)
great_mse = min(meanSE, key=meanSE.get)
great_rmse = min(rmseSE, key=rmseSE.get)

#Setting for determining the worst model based on accuracy, Mean Squared Error and Root Mean Squared Error
low_acc = min(accuracies, key=accuracies.get)
low_mse = max(meanSE, key=meanSE.get)
low_rmse = max(rmseSE, key=rmseSE.get)

#Print the output
print('Best Model based on Accuracy: ', great_acc)
print('Worst Model based on Accuracy: ', low_acc)
print("\n")
print('Best Model based on Mean Squared Error: ', great_mse)
print('Worst Model based on Mean Squared Error: ', low_mse)
print("\n")
print('Best Model based on Root Mean Squared Error: ', great_rmse)
print('Worst Model based on Root Mean Squared Error: ', low_rmse)
```

Best Model based on Accuracy: KNN
Worst Model based on Accuracy: Logistic Regression

Best Model based on Mean Squared Error: KNN
Worst Model based on Mean Squared Error: Logistic Regression

Best Model based on Root Mean Squared Error: KNN
Worst Model based on Root Mean Squared Error: Logistic Regression

Figure (32): Final Determination of Best and Worst Model based on Accuracy, MSE and RMSE

From the **Figure (32)** above, this is where to show a summary of the all models to provide Final Determination of Best and Worst Model based on Accuracy, Mean Squared Error and Root Mean Squared Error. To determine the best model overall, the model should have the highest accuracy and the lowest for both Mean Squared Error and Root Mean Squared Error and vice versa for the worst model. Hence, we can conclude that the best model overall is K-Nearest Neighbour followed by the Logistic Regression as the worst Model overall.

6.0 Deployment

To deploy our KNN model, we will save the model as a pickle object in python and load the model to create a system tool and will help users to estimate and predict the results of the machine, whether or not the machine will give a result of 'OK' or 'NG' and this can be determined by printing out the accuracy of the pickle model as shown below. The model will be stored in the local directory named "Pickle_KNN_Model.pkl" where this source code of (.ipynb) is located. The pickle implementation code can be found below in figure 33(a).

```
In [35]: #We will store this KNN model into pickle format into local directory
#where this .ipynb file located
import pickle
Pkl_nameFile = "Pickle_KNN_Model.pkl"

# Save the Modle to file in the current working directory
with open(Pkl_nameFile, 'wb') as file:
    pickle.dump(knnApr, file)

In [36]: # Load the Model back from file
with open(Pkl_nameFile, 'rb') as file:
    Pickled_KNN_Model = pickle.load(file)
Pkl_nameFile

Out[36]: KNeighborsClassifier(n_neighbors=8)

In [37]: # Use the Reloaded Model to
# Calculate the accuracy score and predict target values

# Calculate the Score
scores = Pickled_KNN_Model.score(dataset_X_test, dataset_y_test)
# Print the Score
print("Test score: {0:.2f} %".format(100 * scores))

# Predict the Labels using the reloaded Model
Ypredicting = Pickled_KNN_Model.predict(dataset_X_test)

Ypredicting

Test score: 99.97 %

Out[37]: array([1, 1, 1, ..., 1, 1, 1], dtype=int64)
```

Figure 33(a): Creating a pickle object and store into the local directory

To deploy our model, we will set the K-Nearest Neighbor as a new prediction model, and initialize it under a function called 'result_predictor'. We will create a main() function to prompt

users to input Point1, Point2, Point3 and Point4 into the system to estimate and forecast whether the machine will provide an 'OK' or 'NG' result which can be found on **Figure (33)** below.

The predictions will be available on demand when they will be able to interact with the prediction model by entering the raw values of Point1, Point2, Point3 and Point4 into the system. The best model for predicting the result from user input is K-Nearest Neighbor (KNN). It is the result we have evaluated. Thus, the system will then analyze the values and correlate them with the days of the week to predict machine results based on KNN model. It can help our users to choose the right time to perform operations on their machine when the probability of the machine result being 'OK' will be higher and thus can save the resources needed to test their machines. The implementation process is as follows:

1. Prompt for user input of 4 values: Point1, Point2, Point3, Point4
2. If user input value is null for any of the input, it will be replaced by 0

DEPLOYMENT

```
#Since the best model overall is the K-Nearest Neighbour (KNN) we can now deploy that model

#Create function for predicting result from getting input from user point1, point2, point3, point4, if no input replace with 0
def result_predictor(Pt1, Pt2, Pt3, Pt4):
    if(Pt1 == ''):
        Pt1 = 0
    if(Pt2 == ''):
        Pt2 = 0
    if(Pt3 == ''):
        Pt3 = 0
    if(Pt4 == ''):
        Pt4 = 0

    predictor = knnApr.predict([[Pt1, Pt2, Pt3, Pt4]])

    if (predictor == [1]):
        pred = 'OK'
    else:
        pred = 'NG'
    return pred
```

Figure 33(b): Function for replace '0' if no input, fit the KNN model and predict results

3. Fit the user input into our best model and predict the input on Point1, Point 2, Point 3 and Point 4 that are given from the user.

```
def main():
```

REVIEWED

By TARUC at 12:45 pm, Aug 08, 2023

```
    print 1 value:")
    print 2 value:")
    Pt3 = input("Enter Point 3 value:")
    Pt4 = input("Enter Point 4 value:")
    machineResult = result_predictor(Pt1, Pt2, Pt3, Pt4)
```

```
print('Final Predicted Machine Result: ', machineResult)
return
```

Figure(34): Function for Getting Input from user

4. Print out the predicted output.

```
main()
```

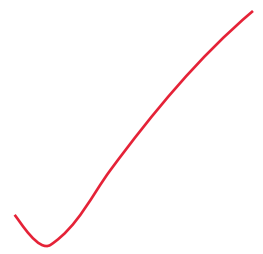
```
Enter Point 1 value:1
Enter Point 2 value:2
Enter Point 3 value:-4
Enter Point 4 value:5
Final Predicted Machine Result: OK
```

Figure(35): For Machine Result 'OK'

```
main()
```

```
Enter Point 1 value:-2945.5
Enter Point 2 value:2358.7
Enter Point 3 value:-1229.4
Enter Point 4 value:-1283.8
Final Predicted Machine Result: NG
```

Figure(36): For Machine Result 'NG'



7.0 Conclusion

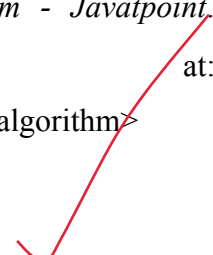
In conclusion, the best model to use on this set of data is by using the **K-Nearest Neighbour**. Even though there are some outliers in the dataset, the K-Nearest Neighbour still manages to predict the best accuracy, lowest Mean Squared Error and lowest Root Mean Squared Error compared to Logistic Regression it produces very bad prediction in terms of accuracy, Mean Squared Error and Root Mean Squared Error and Support Vector Machine produces very optimal prediction in terms of accuracy, Mean Squared Error and Root Mean Squared Error.

Since we know that the best model is K-Nearest Neighbour now we can propose and deploy it to the Hotayi Electronic (M) Sdn. Bhd for their machine failure prediction purposes. It is because even when deployed, the model is still able to predict the correct result with an accuracy of 99.97% when the user enters the points from 0 to 4 which was prompted by the system. We really learned a lot and enjoyed doing this assignment throughout this assignment and as final year students this elective course Data Science is a great opportunity to be learned and implemented in the real world because the current world is fully data driven. This assignment taught us clearly on how to analyse data, preprocess data, fitting into model and evaluate and conclude the project. We hope in the future we can use more dataset, handle missing values, make use of more columns for analysis purposes and even enhance the deployment method by creating an application that lets users enter different points and dates and predict the correct machine result.

7.1 Advantages and disadvantages of Logistic Regression, KNN and SVM

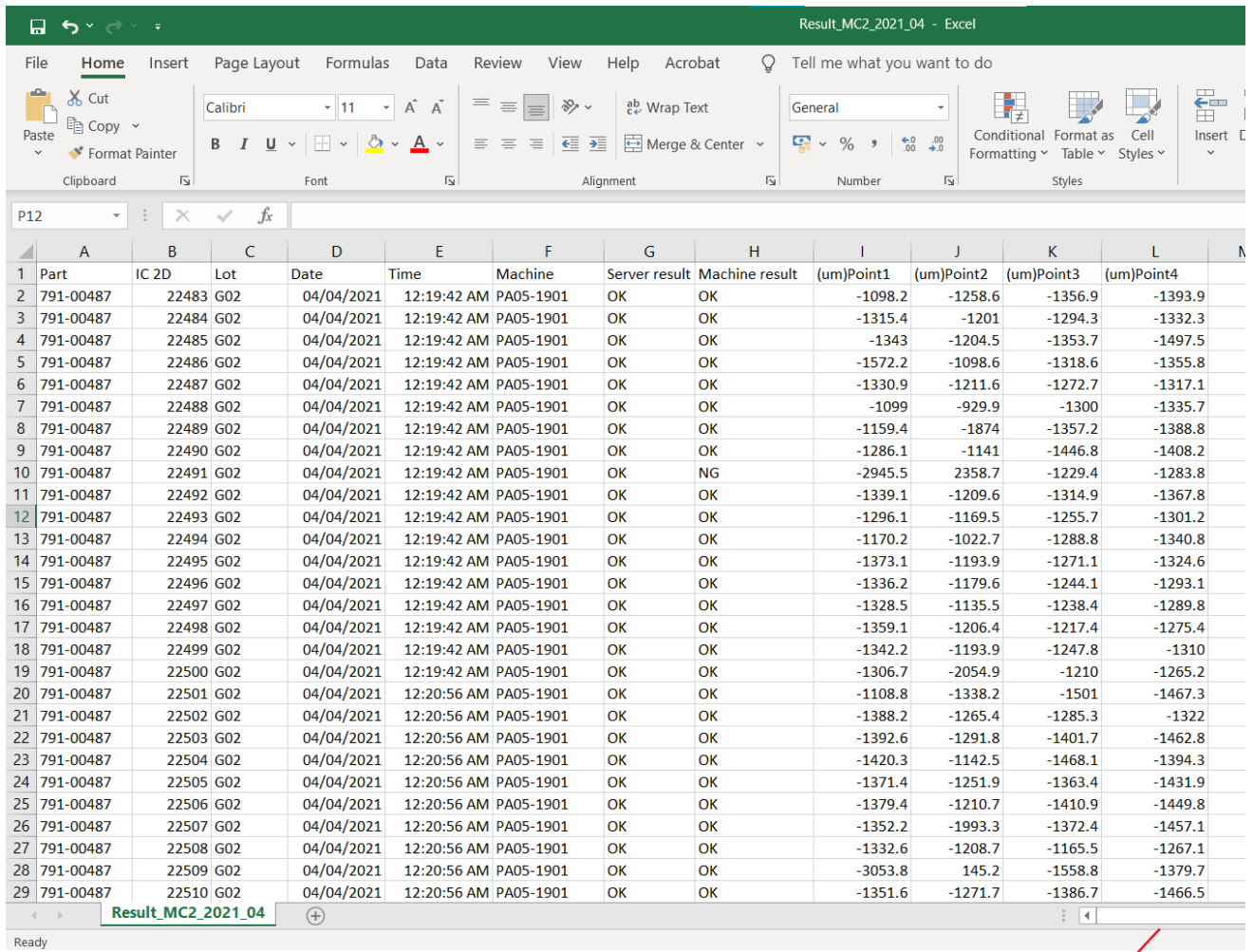
Logistic regression model can be easier to interpret and very efficient to train the data but if the number of data is less than the number of features, it should not be used because it may overfit the data. **K-Nearest Neighbor** model is also a model that is simple and easy to implement. It is a non-parametric algorithm which means it requires no assumptions to implement. On the other hand, this model does not work well with large datasets and is very sensitive to missing value and outliers. Lastly, the good thing about **Support Vector Machine** is that it can work particularly well when it comes with both regression and classification problems. Compared to the other two models, SVM is a difficult model to understand and hard to interpret.

8.0 Reference

- I. Smart Vision Europe. 2021. *Crisp DM methodology - Smart Vision Europe*. [online] Available at: <<https://www.sv-europe.com/crisp-dm-methodology/>> [Accessed 27 August 2021].
 - II. Mayo, M., 2019. *7 Steps to Mastering Data Preparation for Machine Learning with Python — 2019 Edition - KDnuggets*. [online] KDnuggets. Available at: <<https://www.kdnuggets.com/2019/06/7-steps-mastering-data-preparation-python.html>> [Accessed 28 August 2021].
 - III. Agarwal, R., 2019. *The 5 Classification Evaluation Metrics Every Data Scientist Must Know - KDnuggets*. [online] KDnuggets. Available at: <<https://www.kdnuggets.com/2019/10/5-classification-evaluation-metrics-every-data-scientist-must-know.html>> [Accessed 28 August 2021].
 - IV. www.javatpoint.com. n.d. *Logistic Regression in Machine Learning - Javatpoint*. [online] Available at: <<https://www.javatpoint.com/logistic-regression-in-machine-learning>> [Accessed 28 August 2021].
 - V. www.javatpoint.com. n.d. *K-Nearest Neighbor(KNN) Algorithm for Machine Learning - Javatpoint*. [online] Available at: <<https://www.javatpoint.com/k-nearest-neighbor-algorithm-for-machine-learning>> [Accessed 28 August 2021].
 - VI. www.javatpoint.com. n.d. *Support Vector Machine (SVM) Algorithm - Javatpoint*. [online] Available at: <<https://www.javatpoint.com/machine-learning-support-vector-machine-algorithm>> [Accessed 28 August 2021].
- 

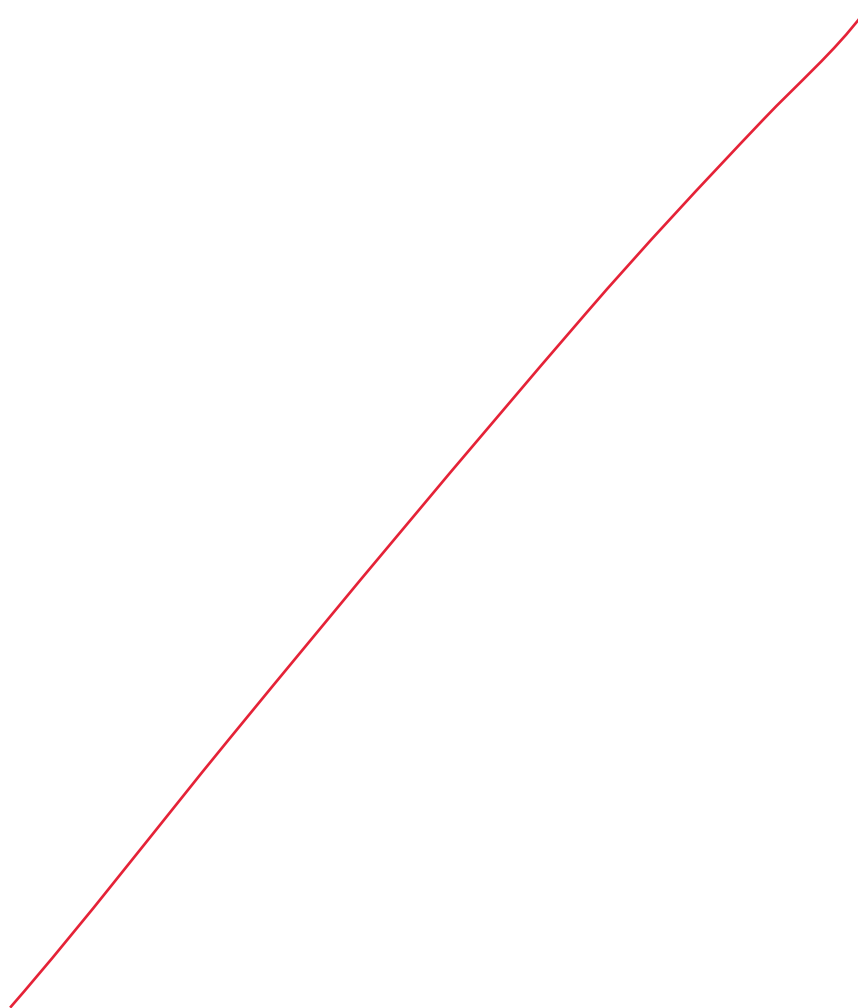
9.0 Appendix

9.1 Dataset



	A	B	C	D	E	F	G	H	I	J	K	L	M
	Part	IC 2D	Lot	Date	Time	Machine	Server result	Machine result	(um)Point1	(um)Point2	(um)Point3	(um)Point4	
1	791-00487	22483	G02	04/04/2021	12:19:42 AM	PA05-1901	OK	OK	-1098.2	-1258.6	-1356.9	-1393.9	
2	791-00487	22484	G02	04/04/2021	12:19:42 AM	PA05-1901	OK	OK	-1315.4	-1201	-1294.3	-1332.3	
3	791-00487	22485	G02	04/04/2021	12:19:42 AM	PA05-1901	OK	OK	-1343	-1204.5	-1353.7	-1497.5	
4	791-00487	22486	G02	04/04/2021	12:19:42 AM	PA05-1901	OK	OK	-1572.2	-1098.6	-1318.6	-1355.8	
5	791-00487	22487	G02	04/04/2021	12:19:42 AM	PA05-1901	OK	OK	-1330.9	-1211.6	-1272.7	-1317.1	
6	791-00487	22488	G02	04/04/2021	12:19:42 AM	PA05-1901	OK	OK	-1099	-929.9	-1300	-1335.7	
7	791-00487	22489	G02	04/04/2021	12:19:42 AM	PA05-1901	OK	OK	-1159.4	-1874	-1357.2	-1388.8	
8	791-00487	22490	G02	04/04/2021	12:19:42 AM	PA05-1901	OK	OK	-1286.1	-1141	-1446.8	-1408.2	
9	791-00487	22491	G02	04/04/2021	12:19:42 AM	PA05-1901	OK	NG	-2945.5	2358.7	-1229.4	-1283.8	
10	791-00487	22492	G02	04/04/2021	12:19:42 AM	PA05-1901	OK	OK	-1339.1	-1209.6	-1314.9	-1367.8	
11	791-00487	22493	G02	04/04/2021	12:19:42 AM	PA05-1901	OK	OK	-1296.1	-1169.5	-1255.7	-1301.2	
12	791-00487	22494	G02	04/04/2021	12:19:42 AM	PA05-1901	OK	OK	-1170.2	-1022.7	-1288.8	-1340.8	
13	791-00487	22495	G02	04/04/2021	12:19:42 AM	PA05-1901	OK	OK	-1373.1	-1193.9	-1271.1	-1324.6	
14	791-00487	22496	G02	04/04/2021	12:19:42 AM	PA05-1901	OK	OK	-1336.2	-1179.6	-1244.1	-1293.1	
15	791-00487	22497	G02	04/04/2021	12:19:42 AM	PA05-1901	OK	OK	-1328.5	-1135.5	-1238.4	-1289.8	
16	791-00487	22498	G02	04/04/2021	12:19:42 AM	PA05-1901	OK	OK	-1359.1	-1206.4	-1217.4	-1275.4	
17	791-00487	22499	G02	04/04/2021	12:19:42 AM	PA05-1901	OK	OK	-1342.2	-1193.9	-1247.8	-1310	
18	791-00487	22500	G02	04/04/2021	12:19:42 AM	PA05-1901	OK	OK	-1306.7	-2054.9	-1210	-1265.2	
19	791-00487	22501	G02	04/04/2021	12:20:56 AM	PA05-1901	OK	OK	-1108.8	-1338.2	-1501	-1467.3	
20	791-00487	22502	G02	04/04/2021	12:20:56 AM	PA05-1901	OK	OK	-1388.2	-1265.4	-1285.3	-1322	
21	791-00487	22503	G02	04/04/2021	12:20:56 AM	PA05-1901	OK	OK	-1392.6	-1291.8	-1401.7	-1462.8	
22	791-00487	22504	G02	04/04/2021	12:20:56 AM	PA05-1901	OK	OK	-1420.3	-1142.5	-1468.1	-1394.3	
23	791-00487	22505	G02	04/04/2021	12:20:56 AM	PA05-1901	OK	OK	-1371.4	-1251.9	-1363.4	-1431.9	
24	791-00487	22506	G02	04/04/2021	12:20:56 AM	PA05-1901	OK	OK	-1379.4	-1210.7	-1410.9	-1449.8	
25	791-00487	22507	G02	04/04/2021	12:20:56 AM	PA05-1901	OK	OK	-1352.2	-1993.3	-1372.4	-1457.1	
26	791-00487	22508	G02	04/04/2021	12:20:56 AM	PA05-1901	OK	OK	-1332.6	-1208.7	-1165.5	-1267.1	
27	791-00487	22509	G02	04/04/2021	12:20:56 AM	PA05-1901	OK	OK	-3053.8	145.2	-1558.8	-1379.7	
28	791-00487	22510	G02	04/04/2021	12:20:56 AM	PA05-1901	OK	OK	-1351.6	-1271.7	-1386.7	-1466.5	

Figure (37): April Dataset MC2_2021_04



Name: Christopher Ewe Kah Thong / Shathiswaran A/L Moorthy

Group: 4

Criterion	Missing or Unacceptable (0-5 mark)	Poor (6-10 marks)	Good (11-15 marks)	Excellent (16-20 marks)	Marks
Data Preprocessing (CLO3)	The data preprocessing step is missing or very poor. Irrelevant, inaccurate, or inappropriate information.	The data preprocessing steps are not listed and described clearly.	The data preprocessing steps are relevant, offering details about the data science components.	The data preprocessing steps introduction are informative, succinct, and offer sufficiently specific details about the data science components.	
Descriptive Analytics and Exploratory Data Analytics (CLO2)	Analytical methods and results are not properly explained. The explanations are not aligned with the data science and business intelligence.	Analytical methods and results are explained. However, the explanations are confusing, incomplete or lacked relevance to the data science and business intelligence.	Analytical methods and results are explained. The explanations are appropriate and related to the data science and business intelligence.	Analytical methods and results are explained well. The explanations are clear, structured, appropriate and related to the data science and business intelligence.	
Graphing and Visualization (CLO2)	Graphing and visualization are omitted or not irrelevant.	Limited graphing and visualization are presented. The information is unclear or confusing.	Graphing and visualization are presented. The information is clear.	Graphing and visualization are presented systematically. The information is comprehensive and informative. All graphs and visualizations are linked to EDA and advanced analytics appropriately.	

Advanced Analytics and Discussion (CLO3)	Discussions are omitted or confusing. No or very little advanced analytics.	Little discussions are presented. Advanced analytics on the results are unclear or confusing.	Discussions of the results are presented. Advanced analytics on the results are presented.	The significance of the results of the work is discussed, sufficiently inclusive of the advanced analytics. Limitations and future improvements of the studies are identified.	
Model Selection and Report Structure (CLO1)	No justification on the model selection. The structure of the report is incomprehensible, irrelevant, or confusing. Transition from one to another is awkward.	Little justifications are presented on the model selection. The structure of the is weak. Transition from one to another is weak and sometimes difficult to understand.	Adequate justifications are presented on the model selection. The structure of the report is good. Transition from one to another is smooth to show the data science concepts.	Strong justifications are presented on the model selection. The structure of the report is excellent. Transition from one to another is smooth and organized to show the data science concepts.	